CONCILIACIÓN DE PAGOS · INTEGRACIÓN

Redis

filtro de duplicados



Quarkus 3.31 · Java 21 · Arquitectura hexagonal

EL PROBLEMA

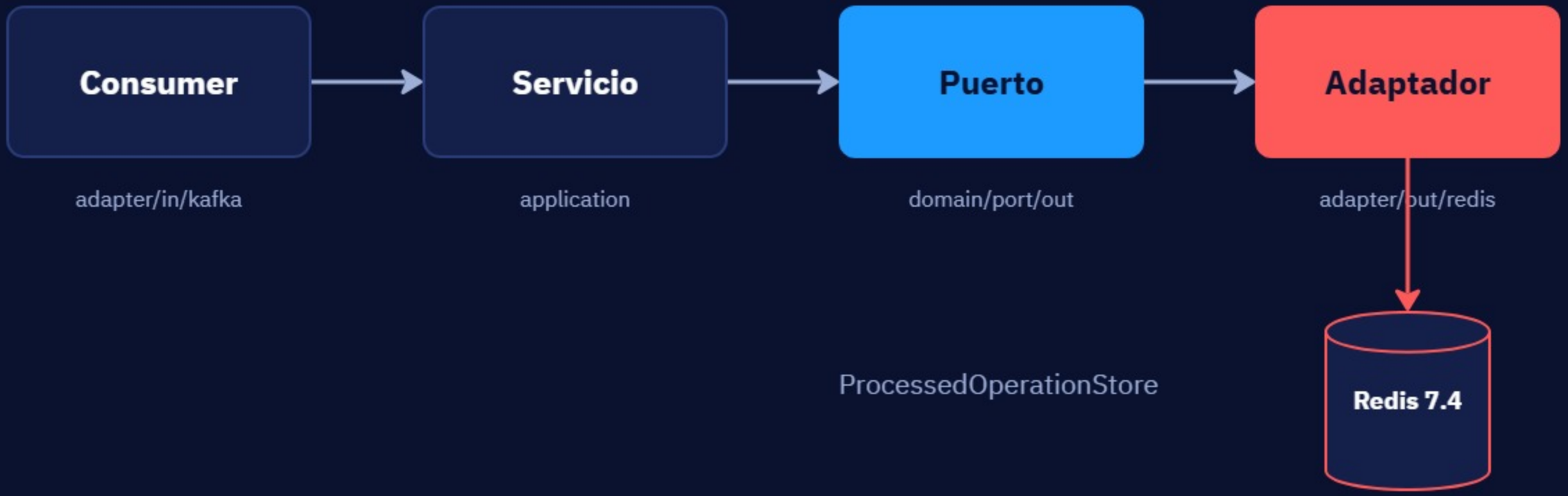
Kafka puede entregar dos veces.



Mismo operationReference, dos entregas.

ARQUITECTURA HEXAGONAL

Redis detrás de un puerto.



EL COMANDO

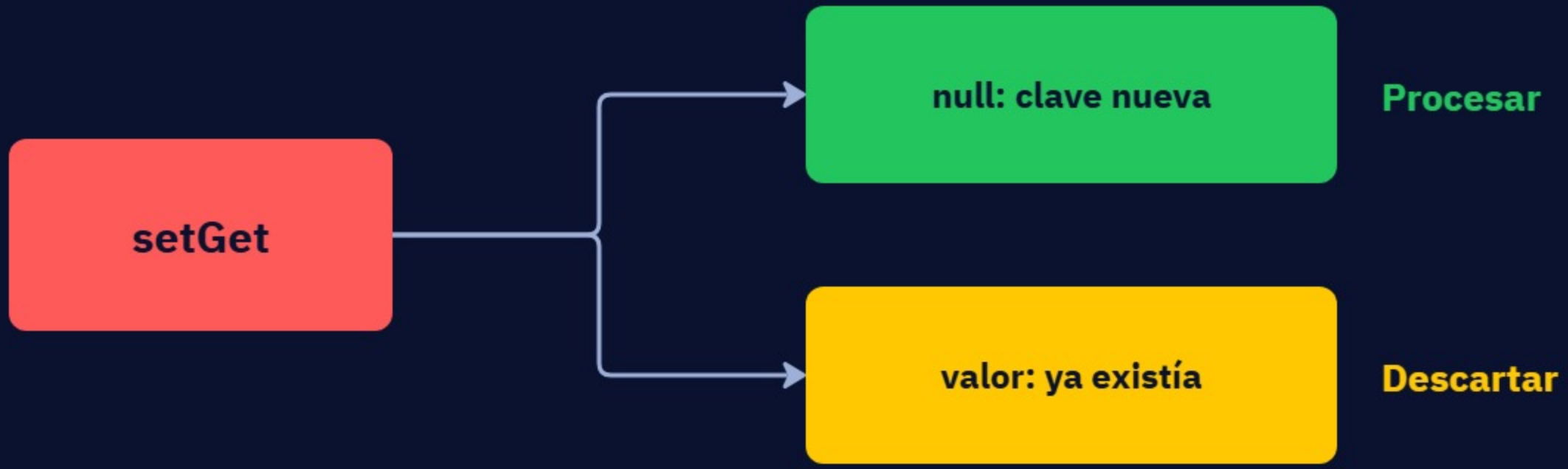
Una sola instrucción.



1 viaje · atómico

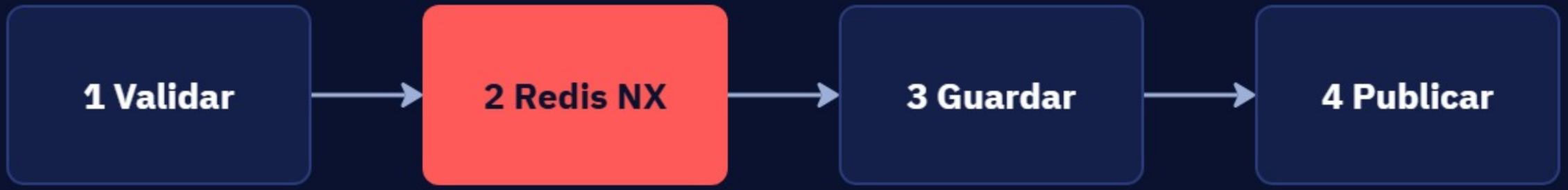
LA RESPUESTA

El valor previo lo dice todo.



FLUJO DE ESCRITURA

Cuatro pasos, en orden.



DEFENSA EN PROFUNDIDAD

Redis acelera. La base garantiza.



TOLERANCIA A FALLOS

Si Redis cae, el flujo sigue.



Disponibilidad antes que optimización

DECISIÓN DE DISEÑO

Por qué SET NX EX GET.

Descartado

SETNX + EXPIRE

2 viajes, no atómico

Descartado

SETEX

No es condicional

Elegido

SET NX EX GET

1 viaje, requiere Redis 7.0 o superior

CONFIGURACIÓN Y ENTORNO

Todo por configuración.

REDIS_HOSTS

variable de entorno

ttl: PT24H

application.yaml

redis:7.4

docker-compose local

Tests

sin Redis, con store en memoria

CONCLUSIONES

Tres ideas para recordar.

1

Filtro, no garantía

2

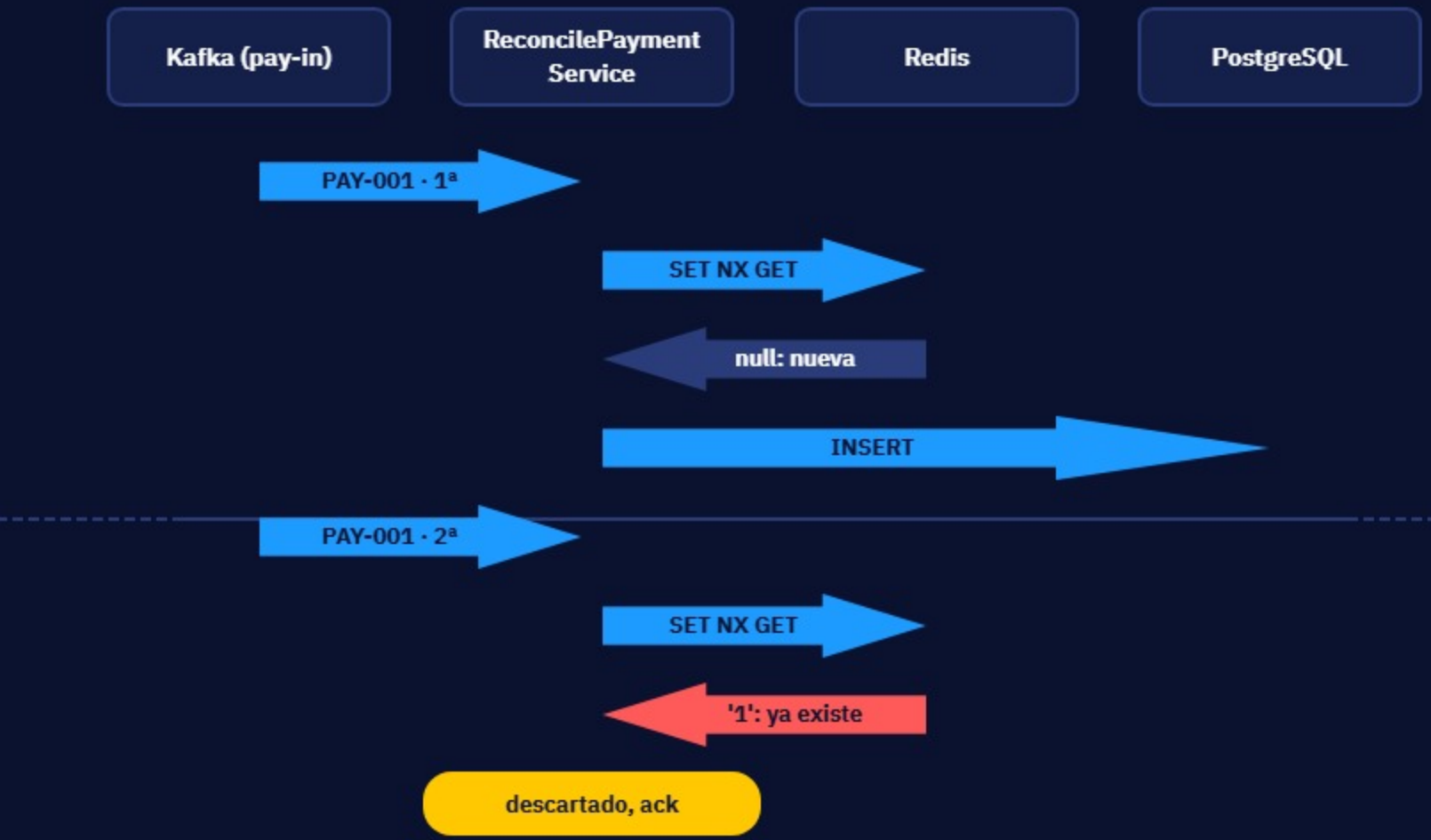
Si cae, sigue

3

Un viaje, atómico

DETALLE · SLIDE 2 · EL PROBLEMA

Dos entregas, un solo efecto.



TEORÍA

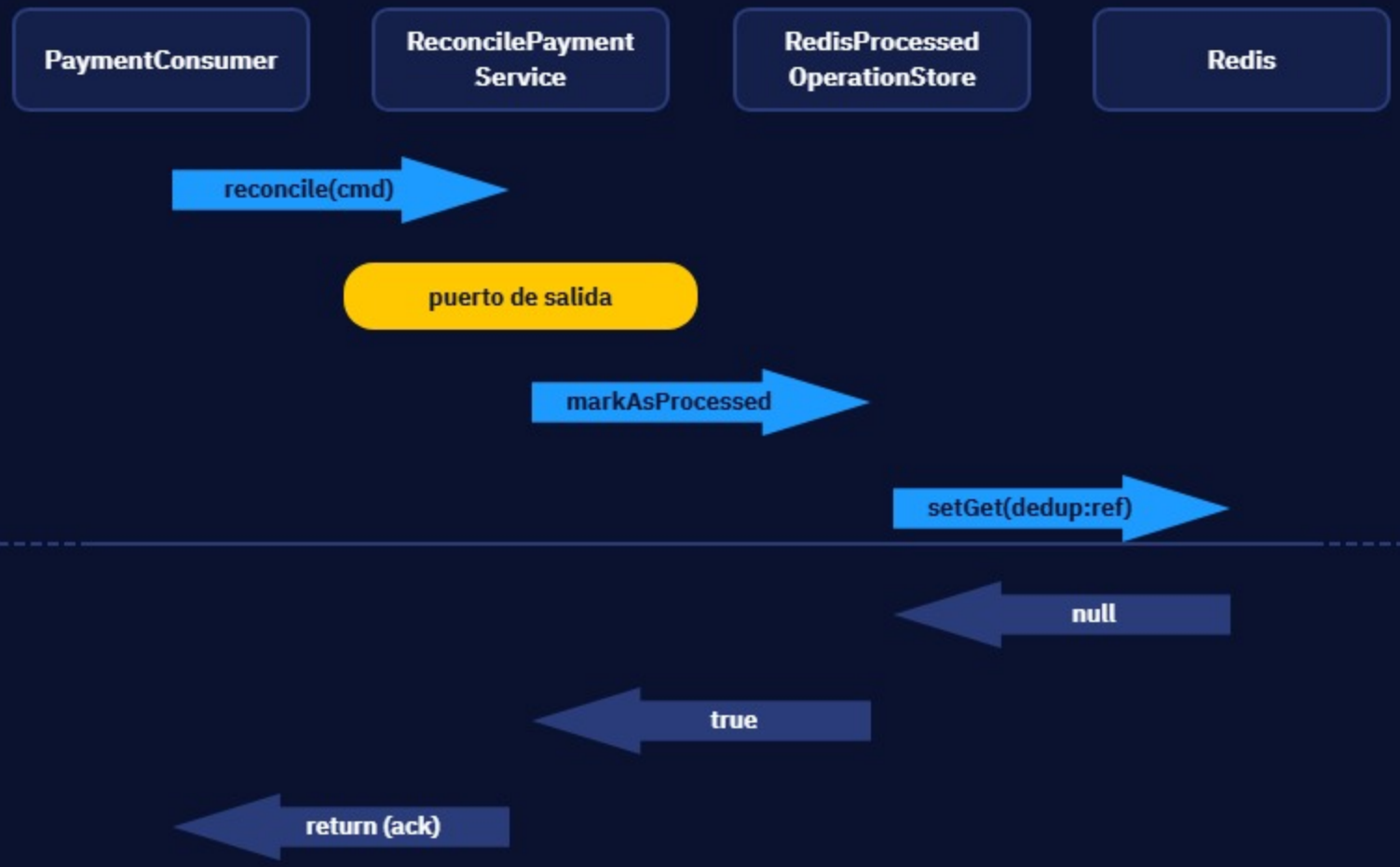
Kafka garantiza entrega al menos una vez, no unicidad: un reintento o un rebalanceo repite el mensaje. Idempotencia es que aplicar la misma operación N veces tenga el efecto de una.

MÉTRICAS

· 2 entregas, 1 solo INSERT
· Clave: operationReference, nunca el id del evento
· Redis se consulta 2 veces; la base se escribe 1

DETALLE · SLIDE 3 · ARQUITECTURA

Redis detrás de un puerto.



TEORÍA

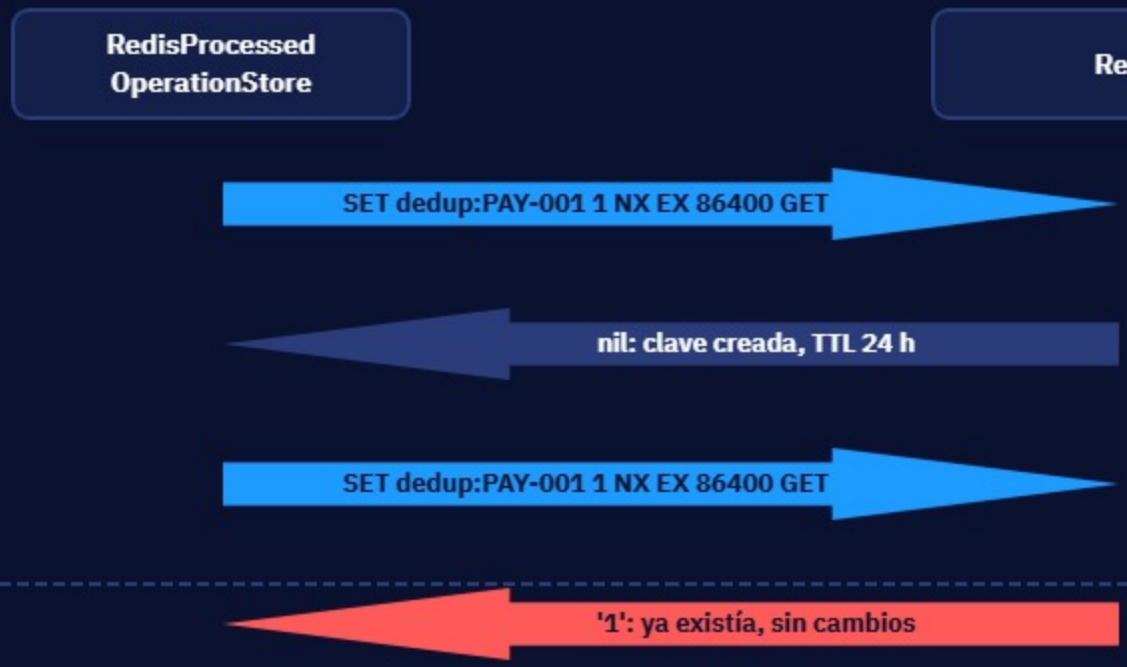
Puertos y adaptadores: el servicio depende de una interfaz del dominio y no conoce Redis. El ImportControl de Checkstyle impide que application importe infraestructura.

MÉTRICAS

- Adaptador: 40 líneas, 1 método público
- Servicio: 67 líneas, sin anotaciones
- En tests, un Set en memoria reemplaza a Redis

DETALLE · SLIDE 4 · EL COMANDO

Una sola instrucción atómica.



TEORÍA

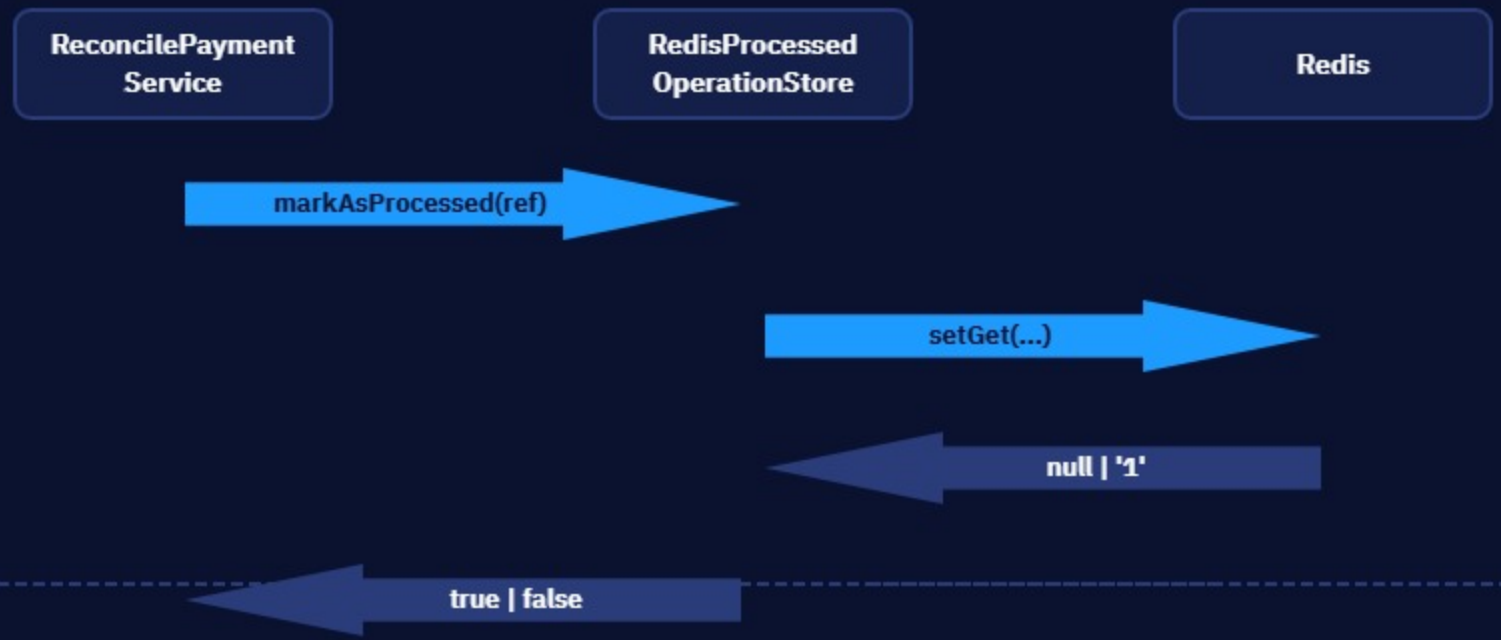
NX crea la clave solo si no existe, EX fija la expiración y GET devuelve el valor anterior (Redis 7.0 o superior). Es un único comando atómico: no hay carrera entre entregas concurrentes.

MÉTRICAS

· 1 round-trip por pago
· TTL: 86 400 s (PT24H)
· Valor de 1 carácter; clave dedup:

DETALLE · SLIDE 5 · LA RESPUESTA

El valor previo decide.



TEORÍA

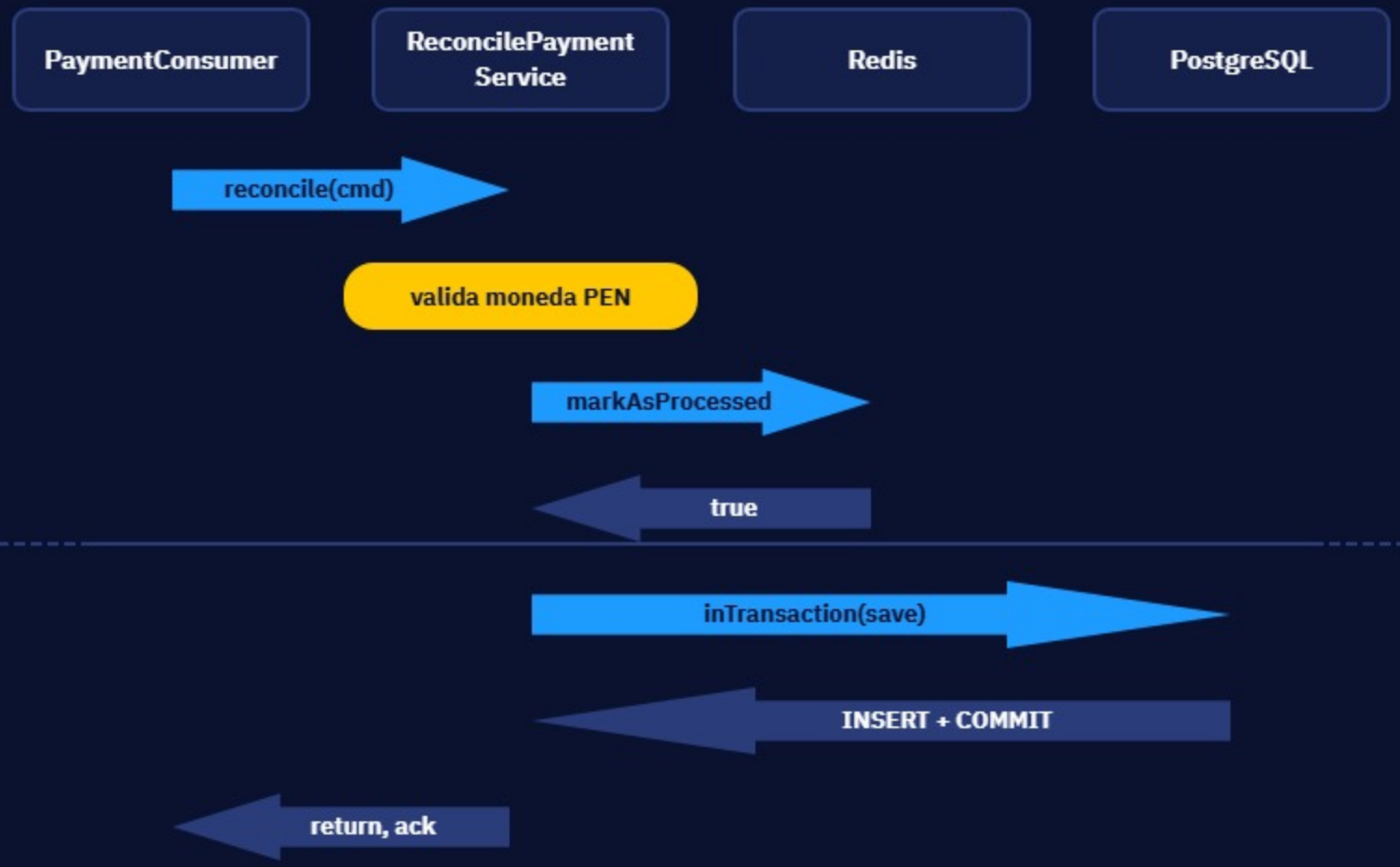
El adaptador traduce valor previo igual a null en el booleano del dominio. Un duplicado se trata como éxito: el mensaje se confirma (ack) y no va a la DLQ.

MÉTRICAS

- Duplicado: 0 escrituras en la base
- 0 mensajes a la DLQ
- Log INFO: Duplicate payment discarded

DETALLE · SLIDE 6 · FLUJO DE ESCRITURA

Validar, filtrar, guardar.



TEORÍA

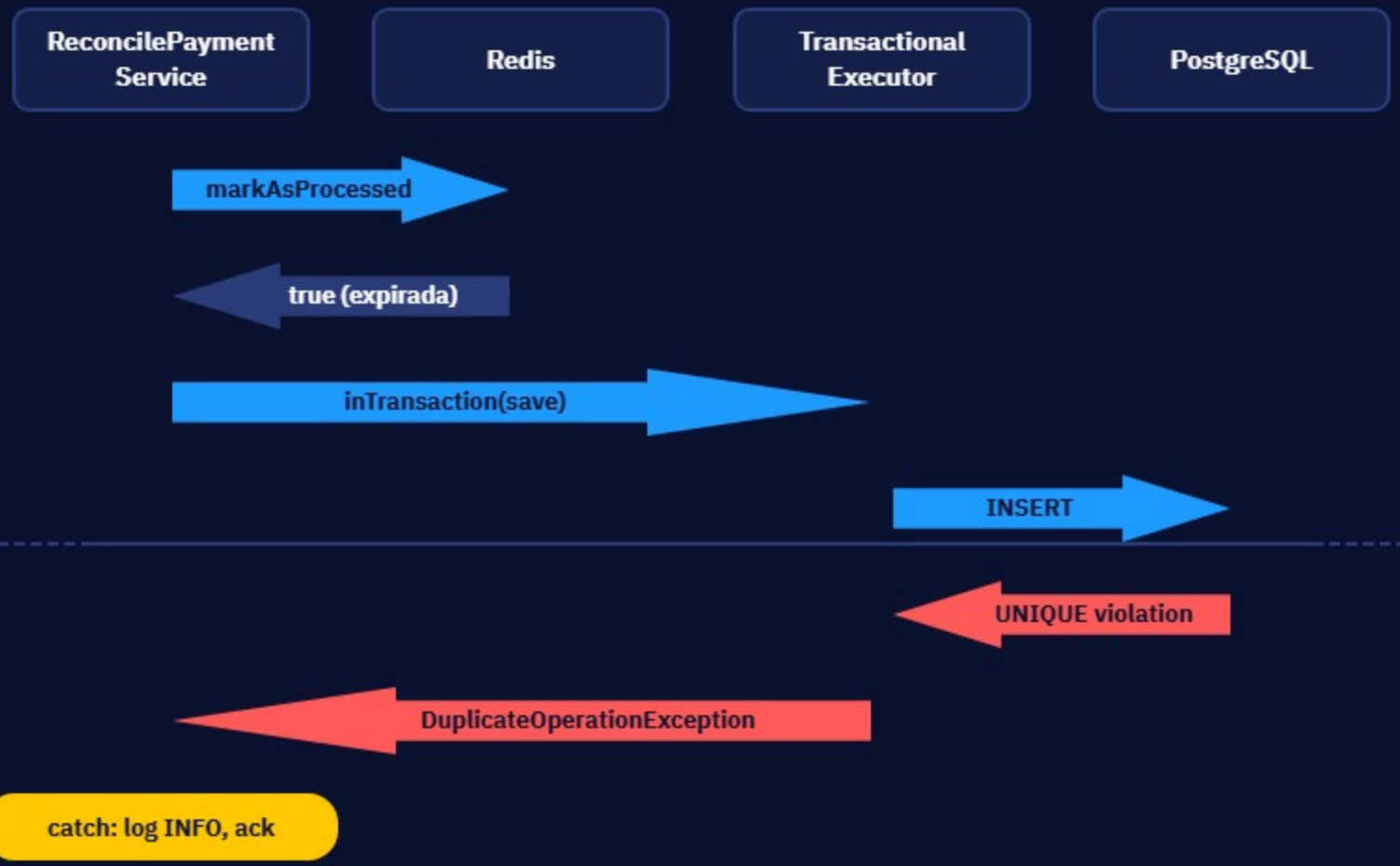
Orden fijo: validar, filtro Redis, guardar dentro de la transacción y publicar después del commit. Los pagos solo crean PENDING; el evento sale al resolver con el movimiento del core.

MÉTRICAS

· 1 llamada a Redis y 1 transacción por pago
· 0 llamadas a Redis en movimientos y en el GET REST
· Concurrencia 1 por canal: orden de llegada

DETALLE · SLIDE 7 · DEFENSA EN PROFUNDIDAD

Redis acelera, la base garantiza.



TEORÍA

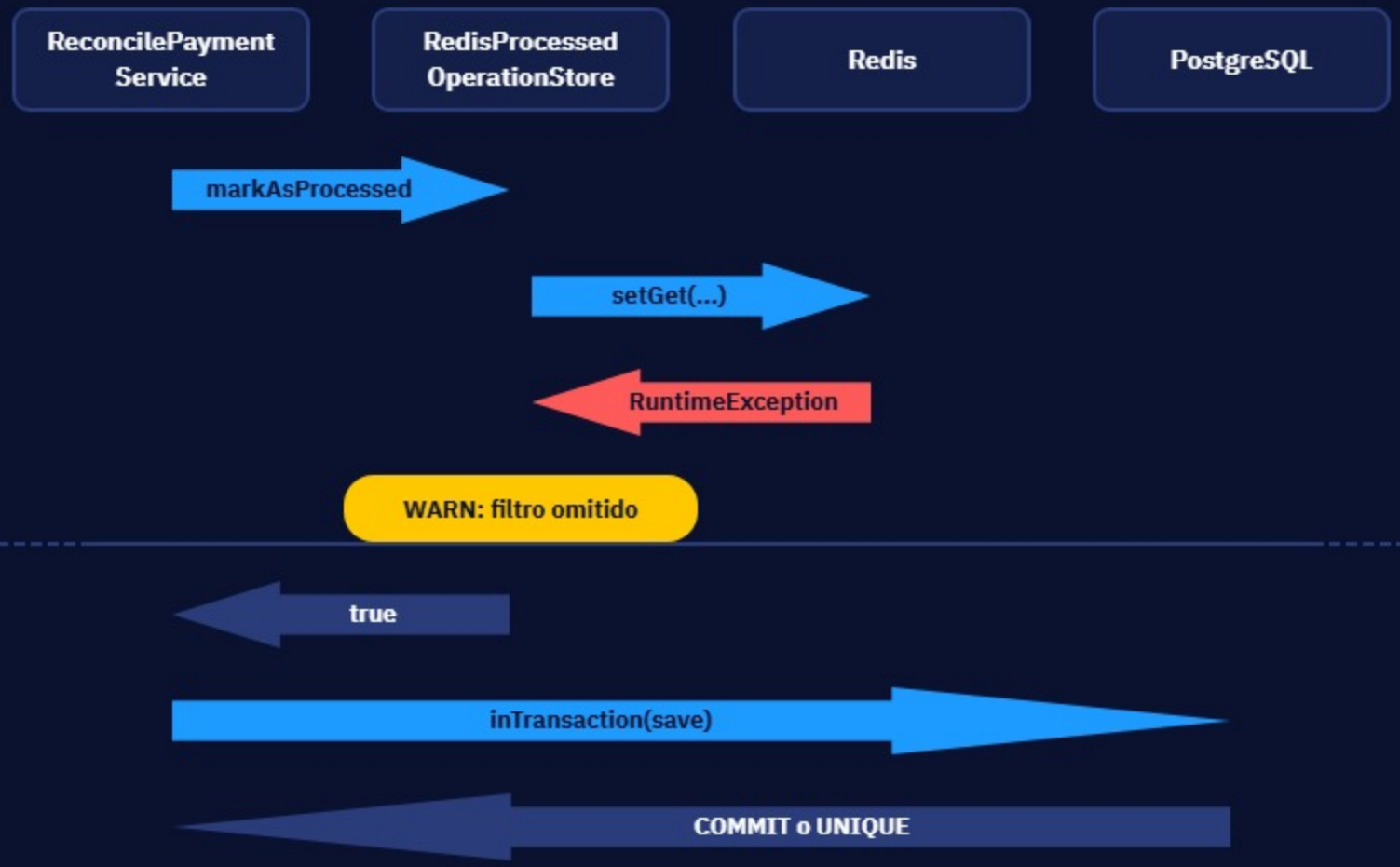
Defensa en profundidad: Redis es la capa rápida y el índice UNIQUE sobre operation_reference es la garantía. Cubre expiración del TTL, reinicios o pérdida de datos en Redis.

MÉTRICAS

- Ventana de Redis: 24 h
- Garantía de la base: sin expiración
- 1 test cubre este camino (unique index)

DETALLE · SLIDE 8 · TOLERANCIA A FALLOS

Si Redis cae, el flujo sigue.



TEORÍA

Fail-open: si el filtro falla se prioriza la disponibilidad. Es seguro porque la corrección no depende de Redis. Fail-closed detendría el consumo de pagos por una dependencia de optimización.

MÉTRICAS

· Log: Redis unavailable, duplicate filter skipped
· Costo: más trabajo para la base mientras dure la caída
· Duplicados: 0 efectos (índice UNIQUE)

DETALLE · SLIDES 9 Y 10 · RESUMEN

Métricas clave.

1
round-trip a Redis por pago

86 400
segundos de TTL (PT24H)

40
líneas del adaptador Redis

5
tests del servicio de pagos, 2 de duplicados

7.0
versión mínima de Redis

Métricas del código y de la configuración. El alcance del proyecto excluye métricas de ejecución.